Sajiya's Sparkle - Integrated Project Report

Project Date: April 2026
Project Version: 1.0.0
Project Status: Deployed

Project Team

Group Members

Serial Number	Name	Student ID	Role
1	Sheikh Hossain Bin Bakhtiar	2023-1-60-060	Team Lead
2	Humaira Habib Usha	2023-1-60-022	Team Member
3	Israt Sultana	2022-2-60-050	Team Member
4	Md Akil Hossain	2023-2-60-091	Team Member
5	Fahim Monaym	2023-1-60-219	Team Member

Table of Contents

PART 1: FORMAL REQUIREMENTS AND SPECIFICATIONS

Chapter	Section	Page
Chapter 1: Introduction		1
	1.1 Background and Motivation	1
	1.2 Problem Statement	2
	1.3 Aim and Objectives	2
	1.4 Project Scope	3
	1.5 Development Methodology	3
Chapter 2: Requirement Collection		4
	2.1 Requirement Collection Approach	4
	2.1.1 Sources of Requirements	4
	2.1.2 Requirement Elicitation Techniques	4
	2.2 Stakeholders and User Roles	5
	2.3 Functional Requirements	5
	2.4 Non-Functional Requirements	6
	2.5 Requirement Prioritization	6

Chapter	Section	Page
Chapter 3: User Stories and Acceptance Criteria		7
	US-01: Browse Products	7
	US-02: Submit a Contact Message	7
	US-03: Register and Log In	7
	US-04: Update Profile	7
	US-05: Add Products to Wishlist	8
	US-06: View and Remove Wishlist Items	8
	US-07: Place an Order	8
	US-08: View My Orders	8
	US-09: Manage Products	8
	US-10: View and Reply to Messages	9
	US-11: View All Orders	9
	US-12: Promote User to Admin	9
	US-13: Demote Admin	9
	US-14: Session Isolation for Dual Access	9
Chapter 4: System Architecture		10
	4.1 Architectural Style	10
	4.2 Main Components	10
	4.2.1 Presentation Layer (Frontend)	10
	4.2.2 Backend Services Layer (Supabase)	10
	4.2.3 Data Storage Layer (PostgreSQL)	10
	4.3 System Interaction and Data Flow	11
	4.4 Security Architecture	11

PART 2: TECHNICAL IMPLEMENTATION

Chapter	Section	Page
Chapter 5: Database Design		12
	5.1 Core Tables and Structure	12
	- Profiles Table	12
	- Products Table	12
	- Orders Table	13

Chapter	Section	Page
	- Order Items Table	13
	- Wishlist Table	13
	- Messages Table	13
	- Signup Rate Limit Table	13
	5.2 Key Database Decisions	14
	5.3 Row Level Security (RLS) Policies	15
Chapter 6: Development Overview		16
	6.1 Module and Component Design	16
	6.2 Design Patterns Used	17
	6.3 Session Management Architecture	18
	6.4 Real-Time Features Implementation	19
Chapter 7: Implementation Challenges and Solutions		20
	Challenge 1: RLS Policy Permission Denied Error	20
	Challenge 2: All Products Showing Stock Quantity Zero	20
	Challenge 3: Admin Dashboard Slow and Manual Refresh	21
	Challenge 4: User and Admin Sessions Interfering	21
	Challenge 5: Phone Number Validation Issues	22
	Challenge 6: Account Deletion DDoS Prevention	22
Chapter 8: Security Implementation Details		23
	8.1 Authentication Flow	23
	8.2 Data Anonymization on Deletion	23
Chapter 9: Technology Stack		24
	Backend Services	24
	Frontend	24
	Development and Deployment	24
Chapter 10: Performance and Optimization		25
	Database Query Optimization	25
	Real-Time Update Latency	25

Chapter	Section	Page
	Frontend Performance	25
Chapter 11: Lessons Learned		26
	1. Session Management Complexity	26
	2. Database Design Impact	26
	3. Real-Time Importance	26
	4. Validation Layering	26
	5. Iterative Problem-Solving	26
	6. Security First	26
Chapter 12: Deployment and Future Enhancements		27
	Current Deployment	27
	Future Enhancements	27
Chapter 13: Conclusion		28
	Key Technical Achievements	28
	Development Methodology	28
	Project Status	28

PART 1: FORMAL REQUIREMENTS AND SPECIFICATIONS

Chapter 1: Introduction

1.1 Background and Motivation

In recent years, online shopping has become one of the most common ways for customers to discover and purchase products. For small businesses and independent brands, having a dedicated online presence is important not only for selling products but also for building customer trust, improving communication, and reaching a wider audience beyond a physical location. However, many small businesses face challenges such as limited technical resources, high development costs, and difficulty maintaining secure and reliable systems.

The Sajiya's Sparkle project was developed to address these needs by creating a modern, web-based storefront that supports product browsing, customer interaction, and order-related functionality while also providing an administrative interface for managing the store's content. The project focuses on delivering a user-friendly experience for customers and a practical workflow for administrators, with strong emphasis on data integrity and security using modern backend services.

1.2 Problem Statement

A typical online store requires more than a visually appealing interface. It must also provide:

- Organized product presentation and categories
- Secure user authentication and profile handling
- Reliable order creation and storage
- Features that improve engagement, such as wishlists
- A communication channel for customer inquiries
- Administrative tools for managing products and messages
- Session isolation for users who need multiple roles (customer and admin)
- Real-time updates to prevent operational lag
- Rate limiting to prevent abuse and DDoS attacks

Without a well-designed system, issues such as inconsistent data, unauthorized access, poor usability, or mutual session interference can reduce customer satisfaction and complicate store operations. Therefore, the core problem addressed by this project is the development of a secure, maintainable, and operationally efficient e-commerce web application that supports both customer-facing and administrative requirements.

1.3 Aim and Objectives

The aim of the Sajiya's Sparkle project is to design and implement a web-based e-commerce platform that enables customers to explore products and interact with the store, while enabling administrators to manage catalog content and customer communication efficiently and securely.

Key Objectives:

- To implement a product catalog that supports browsing by category and viewing product details
- To integrate secure authentication and profile management for users with input validation
- To design a structured database for products, users, orders, wishlists, and customer messages
- To support order processing by storing orders and order items in a consistent relational model
- To implement a wishlist feature with database-level constraints to prevent duplicates and control limits
- To provide an admin interface for managing products and handling customer messages
- To enforce access control using backend policies to protect sensitive data
- To create a responsive and user-friendly UI suitable for both customers and admins
- To implement real-time updates to prevent manual refresh requirements

- To implement session isolation so admin and customer interfaces don't interfere with each other
- To add rate limiting mechanisms to prevent abuse

1.4 Project Scope

The project scope includes the development of a functional online storefront and a supporting backend database. The main features include:

Customer-facing functionality: * Viewing and browsing products * Filtering products by category and collections * Saving products to a wishlist * Submitting messages through a contact form * User authentication and profile updates * Order placement and order history viewing

Administrative functionality: * Managing the product catalog with image uploads * Monitoring and responding to customer messages * Viewing order-related information * Managing admin roles and permissions * Real-time dashboard updates * Rate-limited account deletion and signup

1.5 Development Methodology

The project follows a practical, iterative web-application development approach, emphasizing incremental implementation of features and continuous validation through testing. The core development approach can be summarized as:

1. **Issue-First Analysis:** Identifying critical problems that affect user experience or security
2. **Iterative Implementation:** Building features in small, testable cycles
3. **Continuous Testing:** Validating each implementation immediately before moving forward
4. **Adaptive Planning:** Adjusting approach based on real-world testing results
5. **Security-First Design:** Applying authentication, authorization, and RLS policies at each stage
6. **Performance Optimization:** Monitoring and improving system responsiveness

---

Chapter 2: Requirement Collection

2.1 Requirement Collection Approach

2.1.1 Sources of Requirements

Requirements were collected from the following sources:

- **Stakeholder needs:** Store owner/admin expectations regarding product management, orders, and customer communication
- **Customer expectations:** Ability to browse products easily, save favorites, and contact the store

- **Domain research:** Common e-commerce best practices and security standards
- **Technical constraints:** Supabase platform capabilities and limitations
- **Real-world testing:** Problems identified during development and testing phases

2.1.2 Requirement Elicitation Techniques

- Scenario-based analysis: Converting real-world workflows into system features
- Feature decomposition: Breaking large goals into smaller implementable features
- Prioritization: Separating essential features from optional enhancements
- Iterative refinement: Updating requirements as design and implementation decisions were made
- Problem-driven development: Identifying issues and creating requirements to solve them

2.2 Stakeholders and User Roles

Stakeholder	Description
Store Owner / Super Admin	Manages admins, products, reviews orders, handles inquiries, highest authority level
Admin	Manages products, reviews orders, responds to customer messages
Authenticated Customer	Can maintain profile, create orders, use wishlists, update contact information
Visitor (Anonymous)	Can browse products, submit contact form messages, view public content

System Roles: * **Anonymous User:** Limited access, mainly browsing and messaging * **Authenticated User:** Can place orders, manage wishlist, update profile, view own orders * **Admin:** Can manage catalog, view all orders, manage messages, cannot add/remove other admins * **Super Admin:** Highest authority, can promote/demote admins, manage all system features

2.3 Functional Requirements

Authentication and Security

- The system shall support user registration and login via Google OAuth
- The system shall maintain a user profile linked to authentication identity
- The system shall allow users to update profile information with proper validation
- The system shall validate phone numbers to ensure Bangladeshi format (11 digits, starting with 01)
- The system shall prevent account deletion spam with 15-minute rate limiting
- The system shall prevent users from signing up with recently deleted email addresses

Catalog and Product Browsing

- The system shall allow anyone to view products without authentication

- The system shall display essential product details (name, image, category, price, description)
- The system shall support category-based browsing and featured products
- The system shall display accurate stock status (in_stock or out_of_stock)
- The system shall prevent adding out-of-stock items to cart

Wishlist Management

- The system shall allow authenticated users to add/remove products to/from a wishlist
- The system shall prevent duplicates in the wishlist
- The system shall enforce a wishlist size limit of 5 items per user

Orders

- The system shall allow users to create orders
- The system shall store order headers and order items with purchase-time prices
- The system shall allow users to view only their own orders and order items
- The system shall preserve order history even after user account deletion

Customer Messages

- The system shall provide a contact form for visitors/customers
- The system shall store messages with type, status, and timestamps
- The system shall enable admin reply workflow with status tracking

Admin Features

- The system shall allow admins to manage products (add/edit/delete)
- The system shall allow admins to upload product images
- The system shall allow admins to view all orders and items in real-time
- The system shall allow admins to view customer messages and reply
- The system shall allow super admins to manage admin privileges with defined constraints

Session Management

- The system shall support independent user and admin sessions
- The system shall prevent logout from one interface from affecting the other
- The system shall properly clear cached session data on logout

2.4 Non-Functional Requirements

Requirement	Description
Security	Enforce authentication, role-based access, and database-level RLS policies
Data Integrity	Enforce foreign keys, constraints, triggers, and correct pricing snapshots
Usability	Responsive and user-friendly UI for

Requirement	Description
	customers; efficient workflows for admins
Performance	Indexed retrieval for queries; real-time updates without page reload
Reliability	Soft delete strategy preserves data; audit trails maintained
Maintainability	Modular structure separating UI behavior, data access, and business logic
Scalability	Support query optimization and real-time subscriptions for multiple concurrent users

2.5 Requirement Prioritization

Must Have: * Product browsing * Authentication with Google OAuth * Orders and order items * Wishlist management * Customer messages * Admin catalog management * Role-based access control via RLS * Session isolation * Rate-limited account deletion * Real-time admin dashboard updates

Should Have: * Featured products * Admin message reply workflow * Phone number validation * Stock status auto-sync * Image storage and management

Could Have: * Advanced product search * Discount system * Payment gateway integration * Detailed analytics dashboard

Won't Have (current version): * Complex recommendation engines * Third-party shipping integration

---

Chapter 3: User Stories and Acceptance Criteria

Core User Stories

US-01: Browse Products

As a visitor, I want to view available products so that I can explore what the store offers.

Acceptance Criteria: * Product list is visible without login * Each product displays: name, image, category, and price * Stock status displays correctly (in stock or out of stock) * The page loads without permission errors for anonymous users

US-02: Submit a Contact Message

As a visitor, I want to send a message to the store so that I can ask questions or request support.

Acceptance Criteria: * Contact form is available without login * Required fields (name, email, type, message) are validated * Message record is stored in database after submission * User receives clear confirmation message on success

US-03: Register and Log In

As a customer, I want to register and log in so that I can access personalized features.

Acceptance Criteria: * User can sign up and log in successfully using Google OAuth * Profile record is created after signup * Profile completion form accepts valid Bangladeshi phone numbers (11 digits, starting with 01) * Authentication state persists correctly during navigation until logout

US-04: Update Profile

As a customer, I want to update my profile details so that my contact and shipping info is saved for future use.

Acceptance Criteria: * Customer can update address and phone successfully * Phone validation enforces Bangladeshi format * Only the logged-in user can update their own profile * Data is stored and visible on page reload

US-05: Add Products to Wishlist

As a customer, I want to add products to a wishlist so I can save items for later.

Acceptance Criteria: * Wishlist actions require authentication * Duplicate items cannot be added (unique constraint enforced) * Wishlist cannot exceed 5 items * UI shows feedback if wishlist limit is reached or item is out of stock

US-06: View and Remove Wishlist Items

As a customer, I want to view my wishlist and remove items so I can manage saved products.

Acceptance Criteria: * Only logged-in user can see their wishlist * Removing an item deletes it successfully * UI updates immediately after removal

US-07: Place an Order

As a customer, I want to place an order so that I can purchase products.

Acceptance Criteria: * Order record is created linked to customer profile * Order items records are created with quantity and purchase-time price * Customer can view created order in order history * Customer cannot create order for different user ID

US-08: View My Orders

As a customer, I want to view my orders so that I can track my purchases.

Acceptance Criteria: * Customers can only view own orders * Each order shows status, total amount, and creation date/time * Order items are visible for customer's orders only

US-09: Manage Products

As an admin, I want to manage products so that the storefront catalog remains up to date.

Acceptance Criteria: * Admin can create new products with required fields * Admin can upload product images and store image URLs * Admin can edit product fields including price, description, featured, and stock quantity * Stock status auto-computes from quantity (qty = 0 means out_of_stock) * Admin can delete products if needed * Non-admin users cannot perform admin operations

US-10: View and Reply to Messages

As an admin, I want to view customer messages and reply so that I can support customers.

Acceptance Criteria: * Admin can view message list and details * Admin can update message status (new/read/replied/archived) * Admin can store replies with reply timestamp * Messages remain viewable for history and audit * New messages appear in real-time without refresh

US-11: View All Orders

As an admin, I want to view all orders so that I can handle fulfillment and customer support.

Acceptance Criteria: * Admin can list all orders * Admin can view order items associated with each order * Orders appear in real-time as they are created * Non-admin users cannot access other users' orders

US-12: Promote User to Admin

As a super admin, I want to promote a user to admin so that store operations can be delegated.

Acceptance Criteria: * Only super admin can perform promotion * Promotion succeeds only if user exists * Maximum 2 additional admins are allowed * Success/failure feedback is returned clearly

US-13: Demote Admin

As a super admin, I want to demote an admin so that access can be revoked.

Acceptance Criteria: * Only super admin can demote * Cannot remove super admin privileges using demote function * Target admin loses admin access after demotion

US-14: Session Isolation for Dual Access

As a user who is both customer and admin, I want my logout from the customer side to not affect my admin access.

Acceptance Criteria: * Logging out from customer profile keeps admin session active * Admin interface continues to work in separate browser tab * Logging out from admin dashboard works independently * No "mutual logout" occurs

---

PART 2: TECHNICAL IMPLEMENTATION

Chapter 4: System Architecture

4.1 Architectural Style

Sajiya's Sparkle follows a client-centric web application architecture supported by Backend-as-a-Service (BaaS). In this design:

- The frontend implements the user interface and application logic for customer and admin operations
- Supabase provides backend services including PostgreSQL database, authentication, storage, and Row Level Security
- This architecture reduces custom server implementation while maintaining security and scalability

4.2 Main Components

4.2.1 Presentation Layer (Frontend)

- Renders storefront (home, collections, product displays)
- Handles user interactions (login, wishlist, messaging, orders)
- Provides admin pages for product and message management
- Communicates with Supabase via JavaScript client SDK

4.2.2 Backend Services Layer (Supabase)

- **Authentication (Supabase Auth):** User registration, Google OAuth, session management
- **Database (PostgreSQL):** Application tables with structured schema
- **Row Level Security (RLS):** Database-level access control based on user identity and roles
- **Storage:** File storage for product images with publicly accessible URLs
- **Realtime:** PostgreSQL change subscriptions for real-time updates

4.2.3 Data Storage Layer (PostgreSQL)

- Supports catalog browsing, customer accounts, orders, wishlist, and messages
- Enforces data integrity through foreign keys and constraints
- Implements business rules through triggers and stored functions
- Maintains historical data for audit trails

4.3 System Interaction and Data Flow

Product Browsing (Public): 1. Visitor opens storefront 2. Frontend fetches products from database 3. RLS policies allow public read access 4. Product list displays with stock status

Authentication and Profile Management: 1. User registers/logs in via Google OAuth 2. User profile created in public.profiles table 3. User can view and update only their profile (enforced by RLS) 4. Session stored in browser with logout flag support

Wishlist Workflow: 1. Authenticated user adds product to wishlist 2. Frontend inserts record in wishlist table 3. Database enforces: no duplicates (unique constraint), maximum 5 items (trigger) 4. User can only view and manage their own wishlist entries

Order Creation Workflow: 1. Authenticated user places order 2. System inserts into orders and order_items tables 3. Purchase-time price snapshot stored in order_items 4. User can only view their own orders (enforced by RLS) 5. Admins can view all orders (enforced by RLS)

Customer Messaging Workflow: 1. Visitor/customer submits message via contact form 2. Message inserted into messages table with status 'new' 3. Admin views and replies; status updated to 'replied' 4. Role-based policies restrict admin-only operations

4.4 Security Architecture

Authentication: * Supabase Auth manages identity verification * Google OAuth provides secure third-party authentication * Sessions stored in browser with proper cleanup on logout

Authorization (Role-Based Access Control): * profile.is_admin flag indicates admin privilege * profile.is_super_admin flag indicates highest authority * RLS policies enforce these roles at database level * Frontend cannot override backend security decisions

Data Protection: * All database access controlled by RLS policies * Sensitive profiles only accessible by owner or admin * Products readable by all, writable only by admins * Orders and wishlist items visible only to owner or admin

Rate Limiting: * signup_rate_limit table tracks recently deleted emails * 15-minute cooldown prevents re-signup after deletion * check_signup_rate_limit() RPC validates signup eligibility * Prevents DDoS-style account spam attacks

Soft Delete Strategy: * Deleted users marked with deleted_at timestamp * Personal data anonymized (email, phone, address set to NULL) * Username changed to 'Deleted User' for transparency * Orders preserved with user_id set to NULL (ON DELETE SET NULL) * Maintains business records and audit trail

Input Validation: * Phone validation enforces Bangladeshi format (11 digits, starting with 01) * Frontend validation provides immediate feedback * Backend validation (via RLS) prevents API bypasses * Constraints prevent invalid data at database level

---

Chapter 5: Database Design

5.1 Core Tables and Structure

Profiles Table

```
- id (UUID, Primary Key, Foreign Key to auth.users)
- email (text, nullable - set to NULL on soft delete)
- username (text)
- phone (text, nullable - set to NULL on soft delete)
- address (text, nullable - set to NULL on soft delete)
- is_admin (boolean, default: false)
- is_super_admin (boolean, default: false)
- deleted_at (timestamp, nullable - marks soft delete)
- created_at (timestamp)
```

Purpose: Stores user profile metadata with role-based access control

Products Table

```
- id (bigint, Primary Key)
- name (text, not null)
- category (text, not null)
- price (numeric, not null)
- image_url (text, not null)
- stock_quantity (integer, default: 1)
- stock_status (text, default: 'in_stock')
- featured (boolean, default: false)
- description (text)
- created_at (timestamp)
- updated_at (timestamp)
```

Purpose: Maintains product catalog with auto-computed stock status

Key Decision: Stock status is auto-computed from quantity (qty = 0 = out_of_stock), not manually set. This prevents data inconsistency.

Orders Table

```
- id (bigint, Primary Key)
- user_id (UUID, Foreign Key to profiles - ON DELETE SET NULL)
- total_amount (numeric, not null)
- status (text, default: 'pending')
- order_status (text, default: 'pending_verification')
- payment_method (text, default: 'cash_on_delivery')
- address (text)
- shipping_address (text)
- delivery_address (text)
- transaction_id (text)
- sender_number (text)
- created_at (timestamp)
```

Purpose: Tracks customer orders with payment and delivery information

Key Decision: ON DELETE SET NULL ensures orders are preserved when user deletes account. Maintains business records and customer service capability.

Order Items Table

```
- id (bigint, Primary Key)
- order_id (bigint, Foreign Key to orders - ON DELETE CASCADE)
- product_id (bigint, Foreign Key to products)
- quantity (integer, default: 1)
- price_at_purchase (numeric, not null)
```

Purpose: Normalizes order structure to avoid redundancy

Key Decision: price_at_purchase stores purchase-time price snapshot, preventing inconsistency if product price changes later.

Wishlist Table

```
- id (bigint, Primary Key)
- user_id (UUID, Foreign Key to profiles - ON DELETE CASCADE)
- product_id (bigint, Foreign Key to products - ON DELETE CASCADE)
- created_at (timestamp)
- Unique constraint: (user_id, product_id)
```

Purpose: Join table for user-product wishlist relationships

Key Decision: Unique constraint prevents duplicates. Trigger enforces maximum 5 items per user.

Messages Table

```
- id (bigint, Primary Key)
- customer_name (text, not null)
- customer_email (text, not null)
- message_type (text, not null)
- message_body (text, not null)
- status (text, default: 'new')
- admin_reply (text)
- replied_at (timestamp)
- admin_notes (text)
- created_at (timestamp)
```

Purpose: Stores customer inquiries with admin response workflow

Constraints: message_type and status limited to specific enum values

Signup Rate Limit Table

```
- id (bigint, Primary Key)
- email (text, not null, unique)
- deleted_at (timestamp, default: now())
- can_retry_at (timestamp, not null)
```

Purpose: Prevents spam account deletion and re-signup

Key Decision: 15-minute cooldown window blocks suspicious account patterns (create, delete, recreate cycle).

5.2 Key Database Decisions

Decision 1: Soft Delete with Data Anonymization

- **What:** Deleted profiles marked with deleted_at, personal data anonymized
- **Why:** Preserves order history for business records while respecting user privacy
- **Implementation:** NULL out email/phone/address, set username to 'Deleted User'

Decision 2: ON DELETE SET NULL for Orders

- **What:** When user deleted, their orders remain with user_id = NULL
- **Why:** Maintains complete order history for customer service and audit trail
- **Prevents:** Loss of historical business data

Decision 3: Auto-Computed Stock Status

- **What:** Stock status computed from quantity, never manually set
- **Why:** Prevents data inconsistency between quantity and status
- **Logic:** quantity = 0 → 'out_of_stock', quantity > 0 → 'in_stock'

Decision 4: Price Snapshots in Order Items

- **What:** price_at_purchase stored at order time, not looked up later
- **Why:** Preserves historical accuracy if product price changes
- **Prevents:** Discrepancies between order history and current pricing

Decision 5: Wishlist Constraints via Trigger

- **What:** Database trigger enforces maximum 5 items per wishlist
- **Why:** Business rule enforced at data layer, not just frontend
- **Prevents:** Users bypassing frontend validation

5.3 Row Level Security (RLS) Policies

Profile Access

- Users can read own profile: `WHERE id = auth.uid()`
- Admins can read all profiles
- Users can update own profile: `WHERE id = auth.uid()`
- Only super admins can modify admin flags

Product Access

- All users can read all products
- Only admins can create products
- Only admins can update products
- Only admins can delete products

Order Access

- Users can read/create only own orders: `WHERE user_id = auth.uid()`
- Admins can read all orders
- Order modifications restricted to admins

Order Items Access

- Users can read items only from own orders
- Admins can read all order items
- Order items created automatically with order

Wishlist Access

- Users can read/write only own wishlist: `WHERE user_id = auth.uid()`
- Trigger enforces 5-item limit
- Admins can read all wishlist data

Message Access

- All users can create messages
- Only admins can read/update messages
- Admin reply and status updates restricted to admins

Admin Management

- Only super admin can execute add_admin() function
- Only super admin can execute remove_admin() function
- Limits enforce maximum 2 additional admins

Chapter 6: Development Overview

6.1 Module and Component Design

The project is implemented as a frontend web application using Vanilla JavaScript, interacting directly with Supabase services. The application is organized into functional modules:

Authentication/Profile Module

- **Files:** user-login.js, profile.js, auth-state.js
- **Responsibilities:** Login/signup flow, profile viewing, profile updates, account deletion
- **Key Features:** Phone validation, rate limit checking, soft delete support

Product Module

- **Files:** index.js, collections.js, admin-products.js
- **Responsibilities:** Product listing, filtering, detail rendering, admin CRUD operations
- **Key Features:** Category filtering, featured products, stock status display, auto-computed status

Wishlist Module

- **Files:** profiles with wishlist components, admin-products.js
- **Responsibilities:** Add/remove wishlist entries, display wishlist, enforce constraints
- **Key Features:** Duplicate prevention, 5-item limit, UI feedback

Order Module

- **Files:** checkout.js, admin-orders.js
- **Responsibilities:** Order creation, order history, admin order viewing
- **Key Features:** Order item creation, purchase-price snapshots, real-time updates

Message Module

- **Files:** contact.js, admin-messages.js
- **Responsibilities:** Contact form submission, admin inbox, reply workflow
- **Key Features:** Message status tracking, admin replies, real-time updates

Admin Dashboard Module

- **Files:** admin-dashboard.js, admin-login.js
- **Responsibilities:** Admin authentication, dashboard overview, real-time subscriptions
- **Key Features:** Real-time order/message updates, page focus triggers

6.2 Design Patterns Used

1. **Layered Architecture**
   - Presentation layer: UI rendering and user interaction
   - Service layer: Supabase API calls and data access
   - Benefit: Maintainability and testability
2. **CRUD Pattern**
   - Used in: Product management, message status, wishlist operations
   - Benefit: Consistent, predictable implementation
3. **Role-Based Access Control (RBAC)**
   - Implemented via RLS policies and role flags (is_admin, is_super_admin)
   - Benefit: Security enforced at database level, not just frontend
4. **Database-Driven Business Rules**
   - Rules enforced via constraints, triggers, and stored functions
   - Examples: Wishlist uniqueness, 5-item limit, admin promotion limits
   - Benefit: Consistency across all interfaces
5. **Reactive State Management**
   - Session state stored in sessionStorage (tab-specific) and localStorage (shared)
   - UI updated based on current session state
   - Benefit: Separation of concerns, simplified testing
6. **Real-Time Subscriptions**

- Supabase Realtime for orders and messages
- Auto-refresh on table changes
- Benefit: No manual refresh needed, better UX

6.3 Session Management Architecture

The Challenge

When a user is both a customer and an admin, they use the same Supabase auth token. Calling signOut() affects the entire session, logging them out of both interfaces simultaneously.

The Solution: Tab-Specific Logout Flags

Storage Strategy: * **localStorage:** Supabase auth token (shared across all browser tabs) * **sessionStorage:** Logout flag `_user_logged_out_for_ui` (tab-specific, isolated per window)

How It Works:

When user logs out from profile page: 1. Set `_user_logged_out_for_ui = true` in sessionStorage (this tab only) 2. Clear session cache in sessionStorage 3. Do NOT call signOut() (keeps auth token intact) 4. Redirect to homepage

When page loads: * Check sessionStorage for logout flag * If flag exists, return null session (show "Login" button) * If flag doesn't exist, use Supabase token from localStorage normally * Admin tab has separate sessionStorage, still sees valid session

Benefits: * Admin and user interfaces fully independent * No mutual logout interference * Works across multiple browser tabs * Leverages browser's built-in storage isolation

6.4 Real-Time Features Implementation

Problem Identified

Admin dashboards required manual page refresh to see new orders and customer messages.

Solution: Supabase Realtime Subscriptions

Implementation (admin-dashboard.js):

```
function setupRealtimeOrdersSubscription() {
    const subscription = supabase.channel('orders')
        .on('postgres_changes', {
            event: '*',
            schema: 'public',
            table: 'orders'
        }, (payload) => {
            console.log('Order update:', payload);
            refreshOrders();
```

```
        })
        .subscribe();
}
```

Features: * Listens to all changes (INSERT, UPDATE, DELETE) on orders table * Auto-refreshes when changes detected * Also triggers refresh when page regains focus * Similar subscriptions for messages table

Query Optimization: * Orders query limited to `.limit(100)` records * Prevents loading entire table * Significantly improves page load time

Results: * New orders appear in dashboard within 1-2 seconds * No manual refresh required * Improved admin workflow efficiency

---

Chapter 7: Implementation Challenges and Solutions

Challenge 1: RLS Policy Causing Permission Denied Error

Problem: Google OAuth users couldn't save profile during signup. Error: "permission denied for table profiles"

Root Cause: RLS policy was checking if user is admin by querying the same profiles table. This created recursive queries that were blocked by Supabase security checks.

Example of Problematic Policy:

```
-- This caused infinite recursion
CREATE POLICY "update_profile" ON profiles
    FOR UPDATE TO authenticated
    USING (
        id = auth.uid() OR
        EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND is_admin =
true)
    );
```

Solution Implemented: * Simplified policy to only check `auth.uid()` match * Moved admin access control to other tables instead * Removed circular dependency checks

Fixed Policy:

```
CREATE POLICY "Users can update own profile" ON profiles
    FOR UPDATE TO authenticated
    USING (id = auth.uid());
```

Result: Profile updates now work correctly during signup with Google OAuth

Challenge 2: All Products Showing Stock Quantity of Zero

Problem: Admin dashboard displayed all products with stock_quantity = 0 and stock_status = 'in_stock'. This was inconsistent and confusing.

Root Cause: Two issues: 1. Database default for stock_quantity was 0 2. Admin was manually setting stock_status independent of quantity, creating mismatches

Solution Implemented: 1. Changed default stock_quantity from 0 to 1 2. Implemented auto-computed stock status from quantity 3. Removed manual stock_status override from admin form 4. Frontend logic: `quantity = 0 ? 'out_of_stock' : 'in_stock'`

Code Implementation:

```
const stockQty = Number(product.stock_quantity ?? 0);
const computedStatus = stockQty === 0 ? 'out_of_stock' : 'in_stock';
const displayStatus = stockQty === 0 ? 'Out of Stock' : 'In Stock';
```

Result: Stock status now automatically matches inventory quantity

Challenge 3: Admin Dashboard Slow and Requires Manual Refresh

Problem: Admin had to reload page to see new orders or customer messages

Root Cause: * Query initially fetched all orders/messages without limit * No real-time synchronization * Manual refresh was only way to get updates

Solution Implemented: 1. Added Supabase Realtime subscriptions for orders and messages tables 2. Limited initial query to 100 records with `.limit(100)` 3. Auto-refresh on table change detection 4. Auto-refresh when browser tab regains focus

Performance Results: * Initial page load time significantly reduced * New orders/messages appear within 1-2 seconds * No manual refresh needed * Better admin workflow experience

Challenge 4: User and Admin Sessions Interfering

Problem: Logging out from user profile also logged out admin interface

Root Cause: Both interfaces share same Supabase auth token in localStorage. Any call to `signOut()` affected entire session.

Attempts Made: 1. **Attempt 1:** Called `signOut({ scope: 'local' })` - Still affected admin session because localStorage is shared 2. **Attempt 2:** Set logout flag in localStorage - Affected all tabs because localStorage is shared

Final Solution: Use sessionStorage instead of localStorage for logout flag * sessionStorage is tab-specific and isolated * Shared Supabase token in localStorage remains untouched * Each tab tracks its own logout state independently * See Section 6.3 for detailed implementation

Result: Admin and user sessions now completely independent

Challenge 5: Phone Number Validation Issues

Problem: Invalid phone numbers being accepted during signup (international formats, too short, incomplete)

Solution Implemented:

Bangladeshi Phone Requirements: * Exactly 11 digits total * Must start with "01" * Followed by 9 more digits

Regex Pattern: `/^01\d{9}$/`

Implementation:

```
const cleanPhone = phone.replace(/[\s\-\(\)]/g, '');
if (cleanPhone.length !== 11 || !/^01\d{9}$/.test(cleanPhone)) {
    showFieldError('profilePhone', 'profilePhoneError',
        'Please enter a valid Bangladeshi phone number (e.g., 01712345678)');
    return false;
}
```

Validation Layers: 1. Frontend validation: Immediate feedback to user 2. Backend validation: Prevents API bypasses 3. Database constraint: Enforces at data layer if needed

Result: Only valid Bangladeshi phone numbers accepted

Challenge 6: Account Deletion DDoS Prevention

Problem: Users could delete and recreate accounts repeatedly, creating spam and wasting resources

Solution Implemented: 15-minute rate limiting on account deletion

Process: 1. User deletes account via delete_user_account() RPC function 2. Email logged to signup_rate_limit table with can_retry_at = now() + 15 minutes 3. When user tries to signup again, check_signup_rate_limit() runs 4. If within 15 minutes, signup blocked with countdown message 5. After 15 minutes, signup allowed again

Database Function:

```
SELECT * FROM signup_rate_limit
WHERE email = user_email
AND can_retry_at > timezone('utc'::text, now());
```

Result: Prevents DDoS-style account spam attacks

Chapter 8: Security Implementation Details

8.1 Authentication Flow

Google OAuth Process: 1. User clicks "Sign in with Google" 2. Redirects to Google authentication 3. User grants permission 4. Redirected back to application with auth code 5. Supabase verifies code and creates session 6. Session stored in browser localStorage

Profile Completion: 1. New users land on profile completion form 2. Required fields: Full name, phone, address 3. Phone validated for Bangladeshi format before submission 4. Rate limit checked (blocks if deleted within 15 mins) 5. Profile data saved via RLS-protected upsert 6. User redirected to collections page

8.2 Data Anonymization on Deletion

Soft Delete Process (delete_user_account PL/pgSQL function):

```
-- Get user email for rate limiting
SELECT email INTO user_email FROM auth.users WHERE id = user_id;

-- Anonymize profile data
UPDATE profiles
SET
    email = NULL,
    username = 'Deleted User',
    phone = NULL,
    address = NULL,
    deleted_at = now()
WHERE id = user_id;

-- Delete wishlist items
DELETE FROM wishlist WHERE user_id = user_id;

-- Log deletion for rate limiting
INSERT INTO signup_rate_limit (email, can_retry_at)
VALUES (user_email, now() + INTERVAL '15 minutes');

-- Delete from auth (cascades to profiles)
DELETE FROM auth.users WHERE id = user_id;
```

Result: * Personal data anonymous in database * Order history preserved for business records * Email blocked for 15 minutes from re-signup * Audit trail maintained

---

Chapter 9: Technology Stack

Backend Services

- **Database:** PostgreSQL via Supabase
- **Authentication:** Supabase Auth with Google OAuth
- **API Access:** PostgREST (auto-generated REST API)
- **Real-time:** Supabase Realtime with PostgreSQL subscriptions
- **Storage:** Supabase Storage for product images
- **Functions:** PL/pgSQL stored procedures and triggers

Frontend

- **Core Framework:** Vanilla JavaScript (ES6+)
- **Build Tool:** Vite
- **Client Library:** @supabase/supabase-js v2.99.1
- **Storage:** LocalStorage (persistent) and SessionStorage (tab-specific)
- **Styling:** CSS with responsive design

Development and Deployment

- **Version Control:** Git and GitHub
- **Hosting:** Vercel (frontend)
- **Build Command:** `npm run build` (Vite)
- **Environment:** Node.js environment

Chapter 10: Performance and Optimization

Database Query Optimization

Orders Query: * **Before:** No limit, retrieved entire orders table * **After:** Limited to 100 most recent records with `.limit(100)` * **Result:** Significantly faster dashboard load time

Indexes Added: * idx_messages_created_at (for sorting messages) * idx_messages_status (for filtering messages) * idx_orders_status, idx_orders_order_status (for filtering orders) * signup_rate_limit_email_idx (for rate limit lookups)

Real-Time Update Latency

- **Average Latency:** New orders appear in admin dashboard within 1-2 seconds
- **Method:** Supabase Realtime PostgreSQL change subscriptions
- **Trigger Points:** Table changes detected automatically, page focus regains

Frontend Performance

- **Architecture:** Vanilla JS without heavy framework overhead
- **Bundle Size:** Minimal (only Supabase SDK as main dependency)
- **First Load:** Static hosting provides fast delivery
- **Subsequent Loads:** Page caching by browser significantly reduces load time

---

Chapter 11: Lessons Learned

1. Session Management Complexity

- Supabase single session per user is a true limitation, not a bug
- Tab-specific storage provides elegant solution
- localStorage/sessionStorage separation is powerful for isolation

2. Database Design Impact

- Soft delete decisions significantly affect business logic
- RLS policies are powerful but require careful design (avoid recursion)
- ON DELETE SET NULL preserves data better than CASCADE

3. Real-Time Importance

- Admin dashboards greatly benefit from real-time updates
- Manual refresh is poor UX
- Supabase Realtime is simple to implement and very effective

4. Validation Layering

- Frontend validation alone insufficient for security
- Backend validation via RLS necessary
- Multiple validation layers catch different types of issues
- Database constraints are final safety net

5. Iterative Problem-Solving

- Solving real problems as they emerge is practical
- Testing immediately reveals implementation issues
- Flexibility to change approach leads to better solutions
- Sometimes simple is better than perfect

6. Security First

- RLS policies at database level more reliable than frontend checks
- Soft delete with anonymization better than permanent deletion
- Rate limiting prevents resource abuse
- Never trust client-side validation alone

Chapter 12: Deployment and Future Enhancements

Current Deployment

Frontend: * Deployed on Vercel * Static files served via CDN * Environment variables configured

Backend: * Supabase managed hosting * PostgreSQL database automatically maintained * Auth and storage integrated

Future Enhancements

Payment Integration: * Integrate BKash and Nagad payment APIs * Verify payment before changing order status * Automated order status updates on successful payment

Email Notifications: * Send order confirmation emails to customers * Notify admins of new messages * Send admin replies to customer email

Inventory Management: * Automatic stock reduction when order placed * Low stock alerts to admins * Supplier integration for automatic reordering

Advanced Analytics: * Sales dashboard with charts and metrics * Customer behavior tracking * Product performance analysis

Mobile App: * Responsive design already in place * Progressive Web App capabilities possible * Native mobile app in future

Scalability Improvements: * Additional database indexes for frequent queries * Caching layer for frequently accessed data * CDN optimization for image delivery * Database query optimization for high traffic

---

Chapter 13: Conclusion

The Sajiya's Sparkle project successfully delivers a modern, secure e-commerce web application supporting both customer and administrative workflows. The system effectively implements:

- **Secure Authentication:** Google OAuth integration with profile validation
- **Complex Session Management:** Tab-specific logout flags for independent sessions
- **Real-Time Operations:** Supabase Realtime subscriptions for admin dashboards
- **Comprehensive Security:** RLS policies, soft delete, rate limiting, input validation
- **Professional Database Design:** Normalized schema with integrity constraints
- **Iterative Development:** Problem-driven approach that addresses real issues

Key Technical Achievements

1. **Session Isolation:** Solved the mutual logout problem through tab-specific storage
2. **Real-Time Dashboards:** Eliminated manual page refresh requirement
3. **Data Preservation:** Implemented soft delete with order history retention
4. **Rate Limiting:** Prevented account deletion spam attacks
5. **Security Layering:** Multiple validation layers ensure data integrity
6. **Performance Optimization:** Query limits and indexes support responsive UI

Development Methodology

The project successfully combined formal requirements gathering and system design with practical, iterative implementation. Key principles applied:

- Issue-first analysis and solution design
- Continuous testing and validation
- Adaptive planning based on real-world testing
- Security-first architecture decisions
- Performance optimization throughout development

The project demonstrates how modern backend-as-a-service platforms like Supabase can enable rapid, secure development of e-commerce applications while maintaining strong data integrity and user experience standards.

Project Status

The system is ready for deployment and operation, with potential for future enhancements in payment processing, analytics, and scalability. The foundation is solid, secure, and maintainable for future growth.

Document Version: 2.0 (Integrated)
Last Updated: April 12, 2026
Total Pages: Comprehensive integrated report
Project Status: Active Development, Ready for Deployment